Software Engineer - Technical Assessment

Assignment: Distributed Document Search Service

Overview

Design and implement a prototype of a distributed document search service capable of searching through millions of documents with sub-second response times. This service should demonstrate enterprise-grade architectural patterns including multi-tenancy, fault tolerance, and horizontal scalability.

Time Expectation

3-4 hours (with AI tool assistance encouraged)

Scenario

Your company needs a document search service that can:

- Handle 10+ million documents across multiple tenants
- Support full-text search with relevance ranking
- Return results in under 500ms for 95th percentile queries
- Handle 1000+ concurrent searches per second
- Provide tenant isolation and security
- Scale horizontally as document volume grows

Deliverables

1. Architecture Design Document

Provide a concise architecture document (2-3 pages max) that includes:

- High-level system architecture diagram showing all major components
- Data flow diagram for indexing and search operations
- Database/storage strategy (explain your choice of search engine, database, cache layers)
- API design with key endpoints and contract examples
- Consistency model and trade-offs
- Caching strategy across different layers
- Message queue usage for asynchronous operations
- Multi-tenancy approach and data isolation strategy

2. Working Prototype

Implement a simplified but functional prototype that demonstrates:

- REST API with at least these endpoints:
  - `POST /documents` - Index a new document
  - `GET /search?q={query}&tenant={tenantId}` - Search documents
  - `GET /documents/{id}` - Retrieve document details
  - `DELETE /documents/{id}` - Remove a document
- Basic multi-tenant support (can be header-based or path-based)
- Search functionality (can use Elasticsearch, PostgreSQL FTS, or any search solution)
- Simple caching layer (Redis, in-memory, etc.)
- Basic rate limiting per tenant
- Health check endpoint with dependency status

Technology Choices: Use any language/framework you're comfortable with. Docker-compose setup is encouraged for multi-service architecture.

3. Production Readiness Analysis

Document what would be required to make this production-ready:

- **Scalability**: How would you handle 100x growth in documents and traffic?
- **Resilience**: Circuit breakers, retry strategies, failover mechanisms
- **Security**: Authentication/authorization strategy, encryption at rest/transit, API security
- **Observability**: Metrics, logging, distributed tracing strategy
- **Performance**: Database optimization, index management, query optimization strategies
- **Operations**: Deployment strategy, zero-downtime updates, backup/recovery
- **SLA Considerations**: How to achieve 99.95% availability

4. Enterprise Experience Showcase

Provide brief examples (1-2 paragraphs each) from your experience on:

- A similar distributed system you've built and its scale/impact
- A performance optimization that resulted in significant improvements
- A critical production incident you resolved in a distributed system
- An architectural decision you made that balanced competing concerns

Evaluation Criteria

We will assess your submission based on:

- **Architectural Thinking**: Quality of design decisions and trade-off analysis
- **Code Quality**: Clean, modular, testable code with proper error handling
- **Scalability Awareness**: Understanding of distributed systems challenges
- **Security Mindset**: Proper handling of multi-tenancy and security concerns
- **Production Maturity**: Realistic assessment of production requirements
- **Communication**: Clear documentation and reasoning

Submission Guidelines

1. **Use AI Tools**: You are encouraged to use GitHub Copilot, ChatGPT, Claude, or any other AI tools.
2. **Code Repository**: Provide a GitHub repository with:
    - Source code with clear README
    - Docker-compose file (if applicable)
    - Sample API requests (Postman collection or curl commands)
    - Architecture diagrams (can be simple markdown/ASCII diagrams)
3. **Documentation**: Single PDF or Markdown file containing:
    - Architecture design
    - Production readiness analysis
    - Experience showcase
    - Brief note on AI tool usage

Bonus Points

- Implementing advanced search features (fuzzy search, faceted search, highlighting)
- Demonstrating blue-green deployment strategy
- Including performance benchmarks from your prototype
- Showing cost optimization strategies for cloud deployment
- Contributing to relevant open source projects (provide links)

Notes

- Focus on demonstrating architectural thinking over complete implementation
- The prototype should be functional but doesn't need production-level polish
- Mock external dependencies where appropriate to save time
- Document assumptions clearly

Remember: We're more interested in your thought process, architectural decisions, and understanding of enterprise-scale challenges than in a perfect implementation. Show us how you think about building systems that last.